

Audio Deepfake Detection API

Frontend Integration Guide

Version 1.0 | For frontend engineers integrating the deepfake detection service

Generated: 2026-06-23 07:55 UTC

Classification: Internal | Audience: Frontend Development Team

Canick(SoftPFI)NewBoxIda4a4e4d6fakappetdcb0mlyeefdamel0e354090.francecentral.azurecontainerapps.

1. Overview

The Audio Deepfake Detection API is a production-grade REST microservice that analyzes audio files and determines whether they contain authentic human speech (bona-fide) or artificially generated deepfake / voice clone audio (spoof). It is designed for easy integration into any frontend application -- dashboard, content moderation UI, media player, or browser extension.

Base URL

```
https://audio-deepfake-detector.reddune-ee354d90.francecentral.azurecontainerapps.io
```

Supported Audio Formats

- WAV, FLAC, MP3, M4A, OGG
- Maximum file size: 50 MB
- Maximum duration: 300 seconds (5 minutes)
- Sample rate: automatically handled (internally resampled to 16 kHz)

Archit

This service uses a three-model ensemble: Wav2Vec2 (HuggingFace transformer for semantic analysis), AASIST (graph-based anti-spoofing for acoustic artifact detection), and a custom dual-branch meta-classifier MLP that fuses their outputs into a single confidence score. Every request also generates XAI attention maps (temporal saliency timeline + heatmap image) and a downloadable PDF forensic report.

2. Endpoint Reference

2.1 Health Check | GET /api/v1/health

Use this endpoint to verify the service is running and all models are loaded before sending audio. Returns load status for Wav2Vec2, AASIST, and the Meta-Classifier.

Method: GET

URL: <https://audio-deepfake-detector.reddune-ee354d90.francecentral.azurecontainerapps.io/>

Content-Type: application/json

Response

```
{
  "status": "ok",
  "wav2vec2_loaded": true,
  "aasist_loaded": true,
  "meta_classifier_loaded": true,
  "device": "cpu",
  "uptime_seconds": 1234.5
}
```

All three *_loaded fields must be true for detection to work
If any model is false, notify operations before sending audio

2.2 Deepfake Detection | POST /api/v1/detect

Upload an audio file for deepfake analysis. This is the primary endpoint. The request must be multipart/form-data with the file in the 'file' field. The response includes the verdict, confidence score, per-model breakdown, XAI attention maps, and a link to download the PDF report.

Method:	POST
URL:	https://audio-deepfake-detector.reddune-ee354d90.francecentral.azurecontainerapps.io/
Content-Type:	multipart/form-data
Body Field:	file (UploadFile, required)

Request Example (cURL)

```
curl -X POST \
  -F "file=@sample_audio.wav" \
  https://audio-deepfake-detector.reddune-ee354d90.francecentral.azurecontainerapps.io/detect
```

Full Response JSON

```
{
  "request_id": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
  "prediction": "spoof",
  "confidence": 0.9498,
  "scores": {
    "wav2vec2": 0.0803,
    "aasist": 0.9907,
    "ensemble": 0.9498
  },
  "process_time_ms": 1477.9,
  "attention_maps": {
    "overall_heatmap": "iVBORw0KGgoAAAANS...",
    "layer_count": 12,
    "timeline": [
      {"start_ms": 0, "end_ms": 160,
       "attention_score": 0.267},
      {"start_ms": 160, "end_ms": 320,
       "attention_score": 0.312}
    ]
  },
  "report_download_link": "/api/v1/report/a1b2c3d4-e5f6-7890-abcd-ef1234567890"
}
```

Response Field Reference

<code>request_id</code>	<code>string</code>	UUID for audit trail and report download
<code>prediction</code>	<code>string</code>	"spoof" (fake) or "bona-fide" (real)
<code>confidence</code>	<code>float</code>	0.0 - 1.0 ensemble confidence. 0 = real, 1 = spoof
<code>scores.wav2vec2</code>	<code>float</code>	Wav2Vec2 model confidence (0 - 1)
<code>scores.aasist</code>	<code>float</code>	AASIST model confidence (0 - 1)
<code>scores.ensemble</code>	<code>float</code>	Final meta-classifier output (0 - 1)
<code>process_time_ms</code>	<code>float</code>	Total inference time in milliseconds
<code>attention_maps.overall_heatmap</code>	<code>string</code>	Base64-encoded PNG heatmap image
<code>attention_maps.layer_count</code>	<code>int</code>	Number of transformer layers analyzed
<code>attention_maps.timeline</code>	<code>array</code>	Per-frame attention scores with timestamps
<code>report_download_link</code>	<code>string</code>	Relative URL to download the PDF report

2.3 Download Report | GET /api/v1/report/{request_id}

Downloads a professional PDF report for the given detection request. The report includes a SYSTEM VERDICT block, Decision Executive Summary table, Multi-Model Signal Breakdown, XAI heatmap with temporal saliency analysis, and a Calibration Guide with confidence bar.

Method: GET

URL: <https://audio-deepfake-detector.reddune-ee354d90.francecentral.azurecontainerapps.io/>

Response Type: application/pdf (binary download)

Response Headers

```
Content-Type: application/pdf
Content-Disposition: attachment; filename="deepfake_report_a1b2c3d4.pdf"
```

3. Frontend Integration -- JavaScript

3.1 File Upload with Fetch API

The simplest way to upload an audio file from a browser or Node.js environment. Use FormData to attach the file and send as multipart/form-data.

JavaScript (Fetch API)

```
const API = "https://audio-deepfake-detector.reddune-ee354d90.francecentral.azurecontainerapps.io/api/v1";

async function detectDeepfake(audioFile) {
  const formData = new FormData();
  formData.append("file", audioFile);

  const response = await fetch(`${API}/detect`, {
    method: "POST",
    body: formData,
    // Do NOT set Content-Type header manually --
    // browser sets it with boundary automatically
  });

  if (!response.ok) {
    const error = await response.json();
    throw new Error(error.detail || "Detection failed");
  }

  return await response.json();
}
```

3.2 File Upload with Axios

JavaScript (Axios)

```
import axios from "axios";

const API = "https://audio-deepfake-detector.reddune-ee354d90.francecentral.azurecontainerapps.io/api/v1";

async function detectDeepfake(audioFile) {
  const formData = new FormData();
  formData.append("file", audioFile);

  const response = await axios.post(`${API}/detect`, formData, {
    headers: { "Content-Type": "multipart/form-data" },
  });

  return response.data;
}
```

3.3 Complete Integration Example

A practical example showing file selection, upload, loading state, error handling, and rendering results.

Full Example

```
const API = "https://audio-deepfake-detector.reddune-ee354d90.francecentral.azurecontainerapps.io/api/v1";

const fileInput = document.getElementById("audioFile");
const resultDiv = document.getElementById("result");
const loadingDiv = document.getElementById("loading");
```

```
fileInput.addEventListener("change", async (e) => {
  const file = e.target.files[0];
  if (!file) return;

  // Validate file type
  const validExts = ["wav", "flac", "mp3", "m4a", "ogg"];
  const ext = file.name.split(".").pop().toLowerCase();
  if (!validExts.includes(ext)) {
    return alert("Unsupported format: " + ext);
  }

  // Validate file size (50 MB)
  if (file.size > 50 * 1024 * 1024) {
    return alert("File exceeds 50 MB limit");
  }

  loadingDiv.style.display = "block";
  resultDiv.innerHTML = "";

  try {
    const formData = new FormData();
    formData.append("file", file);

    const resp = await fetch(`${API}/detect`, {
      method: "POST",
      body: formData,
    });

    if (!resp.ok) {
      const err = await resp.json();
      throw new Error(err.detail || `HTTP ${resp.status}`);
    }

    const data = await resp.json();
    renderResult(data);
  } catch (err) {
    resultDiv.innerHTML = `

Error: ${err.message}</div>`;
  } finally {
    loadingDiv.style.display = "none";
  }
});

function renderResult(data) {
  const isSpoof = data.prediction === "spoof";
  const pct = (data.confidence * 100).toFixed(2);

  resultDiv.innerHTML = `
    <div class="verdict ${isSpoof ? 'spoof' : 'bonafide'}">
      ${isSpoof ? "SPOOF (FAKE)" : "BONA-FIDE (REAL)"}
    </div>
    <div class="confidence">
      Confidence: ${pct}%
    </div>
    <div class="scores">
      <div>Wav2Vec2: ${((data.scores.wav2vec2 * 100).toFixed(1))}%</div>
      <div>AASIST: ${((data.scores.aasist * 100).toFixed(1))}%</div>


```

```
    <div>Ensemble: ${data.scores.ensemble * 100}.toFixed(1)}%</div>
  </div>
  <div class="meta">
    Request ID: ${data.request_id.slice(0, 8)}...
    Latency: ${data.process_time_ms} ms
  </div>
  `;
}
```

3.4 Downloading the PDF Report

The response includes a `report_download_link` field. Use it to let users download or view the PDF report.

PDF Download Options

```
const API = "https://audio-deepfake-detector.reddune-ee354d90.francecentral.azurecontainerapps.io/api/v1";

// After receiving detect response (data.report_download_link):

// Option A: Direct download via anchor click
function downloadReport(link) {
  const a = document.createElement("a");
  a.href = API.replace("/api/v1", "") + link;
  a.download = "deepfake_report.pdf";
  a.click();
}

// Option B: Open in new tab
function viewReport(link) {
  window.open(API.replace("/api/v1", "") + link, "_blank");
}

// Option C: Fetch as blob (for display in an iframe)
async function fetchReport(link) {
  const resp = await fetch(API.replace("/api/v1", "") + link);
  const blob = await resp.blob();
  const url = URL.createObjectURL(blob);
  document.getElementById("pdfViewer").src = url;
}
```

3.5 Rendering the XAI Heatmap

The `overall_heatmap` field is a base64-encoded PNG image. Decode it and display as an inline image in the browser.

Display Heatmap

```
function renderHeatmap(base64png) {
  const img = document.createElement("img");
  img.src = "data:image/png;base64," + base64png;
  img.alt = "XAI Attention Heatmap";
  img.style.width = "100%";
  img.style.maxWidth = "600px";
  document.getElementById("heatmap").appendChild(img);
}

// Usage:
renderHeatmap(data.attention_maps.overall_heatmap);
```

3.6 Attention Timeline Chart

The timeline array contains per-frame attention scores. You can plot this as a line chart using any charting library.

Chart.js Timeline

```
// Example with Chart.js
function renderTimeline(timeline) {
  const labels = timeline.map(t =>
    (t.start_ms / 1000).toFixed(1) + "s"
  );
  const scores = timeline.map(t => t.attention_score);

  new Chart(document.getElementById("chart"), {
    type: "line",
```

```
data: {
  labels,
  datasets: [{
    label: "Attention Score",
    data: scores,
    borderColor: "crimson",
    backgroundColor: "rgba(220, 20, 60, 0.1)",
    fill: true,
    tension: 0.4,
  }],
},
options: {
  responsive: true,
  scales: {
    y: { min: 0, max: 1, title: { display: true, text: "Score" } },
    x: { title: { display: true, text: "Time (s)" } },
  },
  plugins: {
    title: { display: true,
      text: "Wav2Vec2 Attention - Temporal Saliency" },
  },
},
});
}
```

4. Interpreting Results

4.1 Verdict Classification

The prediction field provides the final verdict. Use the confidence score to gauge reliability. The ensemble score is the final output.

0.00 - 0.45	bona-fide	Authentic human voice -- accept
0.45 - 0.55	uncertain	Decision boundary -- flag for manual review
0.55 - 1.00	spoof	Deepfake detected -- reject or escalate

4.2 Per-Model Score Breakdown

Each model in the ensemble captures different spoofing signals:

- AASIST:** Sensitive to acoustic artifacts (synthetic tones, vocoder fingerprints, re-encoding noise)
- Wav2Vec2:** Analyzes semantic token distribution and linguistic coherence. Less sensitive to clean TTS but catches lexical anomalies.
- Ensemble:** Learns when to trust each sub-model based on feature patterns. The final decision authority.

4.3 X

The timeline array provides per-frame attention scores (0 - 1). High attention regions indicate where the Wav2Vec2 model focused during inference. Peaks at unexpected time locations (e.g., mid-sentence spikes) often correlate with synthetic artifacts. The heatmap image visualizes this as a color-coded temporal saliency chart.

5. Error Handling

The API returns standard HTTP status codes. Always check the response status before parsing the body.

400	Bad Request	Unsupported file format. Use .wav, .flac, .mp3, .m4a, .ogg
400	No File	The 'file' field is missing from the multipart upload
413	File Too Large	File exceeds the 50 MB limit. Compress or trim before upload
422	Validation Error	Invalid request format. Check multipart structure
500	Internal Error	Model inference failed. Retry; if persistent, check /health
404	Report Not Found	Report ID has expired or never existed. Regenerate via /detect

5.1 Frontend Error Handling Pattern

Robust Error Handling

```
async function safeDetect(file) {
  try {
    const resp = await fetch(`${API}/detect`, {
      method: "POST",
      body: buildForm(file),
    });

    // Handle non-2xx responses
    if (resp.status === 413) {
      return { error: "File too large (max 50 MB)" };
    }
    if (resp.status === 400) {
      const err = await resp.json();
      return { error: err.detail || "Invalid file" };
    }
    if (!resp.ok) {
      return { error: `Server error (${resp.status})` };
    }

    return await resp.json();
  } catch (err) {
    // Network error (offline, DNS, timeout)
    return { error: "Network error -- check your connection" };
  }
}
```

6. Best Practices

6.1 Performance Expectations

Typical inference time: 800 - 2000 ms for a 5-second audio on CPU

First request may be slower if model is cold-starting

The service scales to zero -- expect ~5s cold start if unused for a while

Loading: Show a loading spinner during detection (especially important for cold starts)

Color: Display verdict with clear color coding (red = spoof, green = bona-fide, yellow = borderline 0.45-0.55)

Caching: Store request_id locally so the PDF report link remains accessible

Manual review: Audio deepfakes are a serious classification -- let operators review borderline cases

1. Before each session, call GET /health to verify all models are loaded
2. Validate file extension and size on the client before uploading
3. Handle 413 (too large) and 400 (unsupported format) gracefully
4. Display loading state while waiting for inference (may take several seconds)
5. Render heatmap image from base64 for visual explainability
6. Provide a download link/button for the PDF report
7. Log request_ids for audit trail and debugging
8. In spoof cases, include a manual override option for operators

6.2 R

6.3 In

7. Quick Reference Card

API ENDPOINTS

GET <https://audio-deepfake-detector.reddune-ee354d90.francecentral.azurecontainerapps.io/health>
POST <https://audio-deepfake-detector.reddune-ee354d90.francecentral.azurecontainerapps.io/detect> (multipart)
GET https://audio-deepfake-detector.reddune-ee354d90.francecentral.azurecontainerapps.io/report/{request_id}

KEY CONSTRAINTS

Accepted audio formats: .wav .flac .mp3 .m4a .ogg
Max size: 50 MB
Max duration: 300 seconds
Typical latency: 800-2000 ms

For questions or support, contact the backend team.